目录

UniRoute — 企业级 AI 模型管控平台

产品需求文档（PRD）Phase 1

一、产品概述

二、Phase 1 功能清单

第一优先级：核心链路（F01–F06）

第二优先级：管理能力（F07–F10）

第三优先级：稳定性与安全（F11–F18）

三、功能需求详细规格

F01: OpenAI 兼容网关

F02: 渠道管理

F03: API Key 管理

F04: 鉴权中间件

F05: 预冻结计费

F06: 调用日志

F07: 管理后台 API

F08: 钱包管理

F09: 模型价格表

F10: 管理后台前端

F11: 限流

F12: Failover + 熔断

F13: 敏感词拦截

F14: 幂等性

F15: Token 估算

F16: 预算告警

F17: 系统设置

F18: 健康检查

四、数据模型

表清单（14 张表）

五、非功能需求

六、部署架构

七、Phase 1 验收清单

核心链路

管理能力

稳定性
八、明确不做的事情（Phase 1 排除）
九、风险登记册

UniRoute — 企业级 AI 模型管控平台

产品需求文档（PRD）Phase 1

> **版本**：Phase 1 — Final **日期**：2026-06-18 **状态**：Approved **来源**：基于 PRD v1.5 + 10 轮顾问讨论 + 6 个竞品分析 + 7 篇前沿研究 + StarToken 对比 **用途**：开发团队的唯一 source of truth。讨论过程、顾问意见见 MAAS PRD讨论/ 文件夹。

一、产品概述

一句话定位：帮企业控制 AI 支出的统一 API 网关。

核心价值排序： 1. **成本管控** — 预冻结防超刷、部门级追踪、Token 对账 2. **本地部署** — 数据不出客户网络 3. **统一接入** — 改 base_url 即可接入 6+ 家模型厂商 4. **企业管理** — 三级权限、审计日志、预算告警

技术路线：基于 OneAPI（Go, MIT）Deep Fork，只保留路由骨架和 provider 适配器，完全放弃与上游同步。模块化单体架构，单个 Go 二进制。

Phase 1 范围：本地部署 MVP，Docker Compose 一键部署。 **目标用户**：企业 IT 管理员（管理后台）+ 开发者（调用 API）。 **天使客户**：3 个，年消费约 ¥240,000/客户。

二、Phase 1 功能清单

第一优先级：核心链路（F01–F06）

没有这些，产品无法运行。

#	功能	依赖
F01	OpenAI 兼容网关	—
F02	渠道管理	—
F03	API Key 管理	—
F04	鉴权中间件	F03

#	功能	依赖
F05	预冻结计费	F01, F03
F06	调用日志	F01

第二优先级：管理能力（F07–F10）

没有这些，管理员无法操作。

#	功能	依赖
F07	管理后台 API	F02, F03
F08	钱包管理	F05
F09	模型价格表	—
F10	管理后台前端	F07

第三优先级：稳定性与安全（F11–F18）

没有这些，生产环境有风险。

#	功能	依赖
F11	限流	F03, F04
F12	Failover + 熔断	F01, F02
F13	敏感词拦截	F01
F14	幂等性	F05
F15	Token 估算	F05
F16	预算告警	F08
F17	系统设置	—
F18	健康检查	—

三、功能需求详细规格

F01: OpenAI 兼容网关

支持的端点：

端点	方法	说明
`/v1/chat/completions`	POST	聊天补全（流式 + 非流式）
`/v1/embeddings`	POST	文本嵌入
`/v1/models`	GET	模型列表

模型路由逻辑： 1. 客户端传 `model="deepseek-v4-pro"` 2. 查 `api_keys.allowed_models` 校验权限 3. 查 `channels` 表找到 `status=active` 的匹配渠道（按 `priority` 升序） 4. 应用 `model_mapping` JSON 映射上游模型名 5. 选择该渠道的下一个可用 `channel_key`（轮询） 6. 转发请求到上游

_uniroute 扩展字段（response body 顶层）：

```
{
    "_uniroute": {
        "request_id": "req_a1b2c3d4",
        "cost": { "amount": 0.00318, "currency": "CNY" },
        "provider": "deepseek",
        "channel": "deepseek-official",
        "ttft_ms": 412,
        "cache_hit": false,
        "fallback": false
    }
}
```

- 流式响应：最后一条 chunk 含 `_uniroute` 字段
- OpenAI SDK 默认忽略未知字段，不影响兼容性

SSE 流式规范：

规范项	值
Content-Type	`text/event-stream`
分隔符	`data: {json}\n\n`
心跳	每 15s 一个 comment 行 `: ping\n\n`
正常结束	`data: [DONE]\n\n`
流式错误	`data: {"error":{...}}\n\n` 后 close
客户端断开	立即终止上游推理 + 按已收到 chunk 结算
余额穿底	每 100 Token 检查，穿底时强制断开 + 补发 [DONE]

Acceptance Criteria： – [ ] POST `/v1/chat/completions` 非流式返回 200 + 完整 usage + `_uniroute.cost` – [ ] POST `/v1/chat/completions` 流式返回 SSE，最后 chunk 含 usage + `_uniroute` – [ ] POST `/v1/embeddings` 返回 200 + embedding 向量 – [ ] GET `/v1/models` 返回当前可用模型列表 – [ ] `model_mapping` 正确映射（客户端传 `gpt-5`，上游收到 `gpt-5-2026-06-01`） – [ ] 每 15s 发送 SSE 心跳 `: ping` – [ ] 流式中途上游断开 → 发送 `data: {"error":{...}}` 后 close

---

F02: 渠道管理

渠道生命周期：

```
active → deprecated（厂商通知下线，仍有流量但显示迁移提示）→ disabled（停止接受请求）
```

渠道 Key 池： – 每个渠道可配置多个上游 Key – Key 使用 AES-256-GCM 加密存储（`ENCRYPTION_KEY`） – 轮询选择下一个可用 Key（round-robin） – Key 连续失败 3 次自动标记为 `invalid`

model_mapping 格式（channels 表 JSONB 字段）：

```
{
    "gpt-5": "gpt-5-2026-06-01",
    "claude-opus": "claude-opus-4-20260514"
}
```

Acceptance Criteria： – [ ] 创建渠道后可测试连通性（调用上游 `/v1/models`） – [ ] 禁用渠道后所有请求走其他渠道 – [ ] Key 加密存储，数据库中看不到明文 – [ ] `model_mapping` 生效 – [ ] 渠道测试页面显示：响应内容 + 费用 + 延迟 + curl 示例 – [ ] `deprecated` 渠道在管理后台显示黄色警告标签 + 推荐迁移渠道

F03: API Key 管理

Key 格式：`ur-{org_id_4位}-{random_32字符}` – 示例：`ur-0001-a3b8f2e1d4c5b6a7e8f9d0c1b2a3e4f5`

存储： – `key_prefix` = 前 12 字符（用于索引） – `key_hash` = HMAC-SHA256(full_key, ENCRYPTION_KEY) – 数据库中不存储明文

查询流程： 1. 从 `Authorization: Bearer <token>` 提取 2. 前缀匹配 `key_prefix` 索引 3. HMAC-SHA256 比对 4. 检查 `status = active` 5. 检查 IP 白名单 6. 检查 `allowed_models`

Acceptance Criteria： – [ ] 创建 Key 后返回完整 Key（只显示一次） – [ ] 用 Key 调用 API 返回 200 – [ ] 禁用 Key 后调用返回 401 – [ ] IP 白名单生效（非白名单 IP 返回 403 + `ip_denied`） – [ ] `allowed_models` 生效（调用非授权模型返回 403 + `model_not_allowed`）

F04: 鉴权中间件

执行顺序（在 relay 之前）： 1. 提取 API Key → 验证 → 查询 org_id 2. 检查 `status = active` 3. 检查 IP 白名单 4. 检查 `allowed_models` 5. 检查 RPM 限流 6. 通过 → 写入 `c.Set("org_id", orgID)` 继续

Acceptance Criteria： – [ ] 无 Key → 401 – [ ] 禁用 Key → 401 – [ ] IP 不在白名单 → 403 – [ ] 模型不在 allowed_models → 403 – [ ] RPM 超限 → 429

F05: 预冻结计费

核心公式：

```
freezeAmount = max(
    (promptTokens × inputPrice) + (max_tokens × outputPrice × safety_factor),
    MIN_FREEZE_AMOUNT
)
```

参数定义： – `promptTokens = ceil(len(prompt_content) / TOKEN_RATIO)` – 中文字符: `TOKEN_RATIO = 1.5` – 英文字符: `TOKEN_RATIO = 4` – 混合: 按中文字符占比加权 – `max_tokens` = 请求中的 `max_tokens`，未传则用模型默认值（如 4096） – `safety_factor` = 1.5（环境变量 `FREEZE_SAFETY_FACTOR`） – `MIN_FREEZE_AMOUNT` = 10 分 = ¥0.10（环境变量 `MIN_FREEZE_AMOUNT`）

生命周期：

```
FREEZE_REQUESTED → FREEZE_SUCCESS → SETTLED
                 → FREEZE_FAILED（余额不足，返回 402）
                              → CANCELLED（请求失败，退还冻结金额）
```

并发控制： – PostgreSQL `WHERE (granted_balance + balance) >= amount` 同时充当余额检查和并发保护 – Redis 缓存热余额（AOF always），快速拒绝明显不足的请求 – 扣费优先级：先扣 `granted_balance` → 再扣 `balance`

流式结算特殊处理： 1. 每收到 100 个 token，检查 Redis 缓存余额 2. 余额穿底 → 强制断开上游连接 + 补发 [DONE] + 按已收到 token 结算 3. 客户端断开（`context.Canceled`）→ 关闭上游

→ 按已收到 chunk 结算 4. 上游断开未返回 usage → 本地 token 计数器强制结算 5. `usage_source` 字段标记：`upstream`（厂商返回）/ `estimated`（本地估算）

Acceptance Criteria： – [ ] 余额充足时请求正常转发并结算 – [ ] 余额不足时返回 402 + `{"error":{"code":"billing_error.insufficient_balance"}}` – [ ] 请求失败时冻结金额全额退还 – [ ] 结算金额 = 实际费用（不超过冻结金额） – [ ] 并发 100 个请求同时冻结，钱包余额不出现负数 – [ ] 流式请求中断后 `call_logs.usage_source = "estimated"`

F06: 调用日志

记录字段： – `request_id`, `org_id`, `api_key_id`, `model`, `provider`, `channel` – `prompt_tokens`, `completion_tokens`, `total_tokens` – `cost_cents`, `latency_ms`, `ttft_ms` – `status_code`, `upstream_status_code`, `upstream_error` – `usage_source`（upstream/estimated） – `is_stream`（boolean）

Acceptance Criteria： – [ ] 每次请求（成功/失败）都写 `call_logs` – [ ] 异步 goroutine 写入（`defer recover`），不阻塞响应 – [ ] 日志写入失败不影响请求处理 – [ ] `call_logs` 支持按 `org_id` + `created_at` 范围查询

F07: 管理后台 API

端点清单（详见 OpenAPI spec）：

模块	端点前缀	操作
认证	`POST /api/v1/auth/login`	登录获取 JWT
组织	`/api/v1/admin/orgs`	CRUD
用户	`/api/v1/admin/users`	CRUD
渠道	`/api/v1/admin/channels`	CRUD + 测试
API Key	`/api/v1/admin/keys`	CRUD + 状态切换
钱包	`/api/v1/admin/wallets`	充值/调账/赠送
价格	`/api/v1/admin/prices`	CRUD + 组织级覆盖
日志	`/api/v1/admin/logs`	查询 + 筛选
设置	`/api/v1/admin/settings`	CRUD

模块	端点前缀	操作
Dashboard	`GET /api/v1/admin/dashboard`	聚合统计

权限模型：

角色	Dashboard	渠道	Key	账单	日志	设置
`platform_admin`	✅	✅	✅	✅	✅	✅
`org_admin`	本组织	❌	本组织	本组织	本组织	❌
`org_viewer`	本组织	❌	只读	只读	只读	❌

Acceptance Criteria： – [ ] JWT 登录/鉴权正常 – [ ] `platform_admin` 可操作所有资源 – [ ] `org_admin` 只能操作本组织资源 – [ ] `org_viewer` 只能查看本组织资源

F08: 钱包管理

操作类型：

操作	说明	幂等性
`recharge`	充值（增加 `balance`）	`client_request_id` 唯一
`adjust`	管理员调账（增减 `balance`）	`client_request_id` 唯一
`grant`	赠送额度（增加 `granted_balance`）	`client_request_id` 唯一

赠送额度特殊规则： – 可设置过期时间 – 扣费优先级：先扣 `granted_balance`（按过期时间近到远）→ 再扣 `balance` – 所有操作写 `audit_logs` + `grants` 表

Acceptance Criteria： – [ ] 充值后余额正确增加 – [ ] 重复 `client_request_id` 不重复充值（幂等） – [ ] 赠送额度可设置过期时间 – [ ] 过期的赠送额度不参与扣费 – [ ] 所有钱包操作写 `audit_logs`

F09: 模型价格表

结构： – `model_prices` 表：全局基础价格（`input_price_cents`，`output_price_cents`） – `org_model_prices` 表：组织级覆盖（`multiplier` 字段，如 1.15 = 加价 15%）

默认加价率：基础 15%，大客户 10%

Acceptance Criteria： – [ ] 可配置每个模型的输入/输出单价 – [ ] 组织级覆盖生效（`org_model_prices.multiplier`） – [ ] 未覆盖的模型使用全局基础价格 – [ ] 价格变更后立即生效（无缓存延迟）

F10: 管理后台前端

页面清单：

页面	路由	说明
Dashboard	`/admin`	总调用量、总费用、活跃 Key 数、余额分布
渠道管理	`/admin/channels`	CRUD + 测试 + Key 池管理
API Key	`/admin/keys`	CRUD + 状态切换 + IP 白名单
账单	`/admin/billing`	费用统计 + 充值记录 + CSV 导出
Token 对账	`/admin/reconciliation`	平台 usage vs 厂商账单对比
调用日志	`/admin/logs`	分页 + 筛选 + 详情
渠道测试	`/admin/test`	选择渠道/模型 → 发送测试消息 → 显示结果
系统设置	`/admin/settings`	全局配置（Webhook、加密密钥等）

技术方案：保留 OneAPI 原生前端（React 18 + Semi Design + Vite），新增页面统一使用 Semi Design 保持风格一致。

Acceptance Criteria： – [ ] 6 个核心页面可操作（Dashboard / 渠道 / Key / 账单 / 日志 / 设置） – [ ] 渠道测试页面显示响应 + 费用 + 延迟 + curl 示例 – [ ] 账单页面支持 CSV 导出 – [ ] Token 对账页面显示平台 vs 厂商差异

F11: 限流

两层限流：

层级	存储	算法	说明
按 Key	Redis	令牌桶	`api_keys.rpm_limit`（默认 60）

层级	存储	算法	说明
按渠道	Redis	令牌桶	`channels.rpm_limit` / `channels.tpm_limit`

Acceptance Criteria： – [ ] Key RPM 超限 → 429 + `rate_limit_error.rpm_exceeded` – [ ] 渠道 TPM 超限 → 自动切换到同模型其他渠道 – [ ] 限流计数器在 Redis 中，服务重启不丢失

F12: Failover + 熔断

重试策略：同模型重试（同厂商多 Key 轮询），不做跨模型 Fallback。 – 原因：跨模型 Fallback 会导致预冻结金额不匹配。

熔断器状态机：



触发熔断的错误：408 超时、500、502、503、连接超时 **不触发熔断的错误**：400（客户端）、401（标记 Key 无效）、429（Key 冷却 60s）

Acceptance Criteria： – [ ] 渠道 A 全部 Key 失败 → 自动切换到同模型的渠道 B – [ ] 连续 5 次失败 → 渠道进入熔断状态 – [ ] 熔断 5 分钟后进入半开放状态 – [ ] 半开放状态只放行 10% 流量 – [ ] 半开放状态成功后恢复 HEALTHY

F13: 敏感词拦截

实现：DFA（确定有限自动机）模式匹配，零外部依赖。

位置：请求到达后、转发前。 **命中**：返回 400 + `validation_error.sensitive_word_detected` **配置**：管理员可通过设置页面管理词库。

Acceptance Criteria： – [ ] 请求包含敏感词 → 400 + 错误信息 – [ ] 词库可配置（增删改） – [ ] 匹配不影响正常请求延迟（<1ms）

F14: 幂等性

存储：`freeze_idempotency` 表，`request_id` 主键。

状态机：`Processing` → `Success` / `Failed`

流程：1. 请求到达，提取 `X-Request-Id` header 2. 查询 `freeze_idempotency` 表 3. 已存在且 `Success` → 直接返回缓存结果 4. 已存在且 `Processing` → 返回 409 Conflict 5. 不存在 → 插入 `Processing` → 正常处理 → 更新为 `Success` / `Failed`

Acceptance Criteria：– [ ] 重复 `X-Request-Id` 返回相同结果（不重复扣费） – [ ] 并发相同 `X-Request-Id` → 一个成功，其余 409 – [ ] `Processing` 状态超时（300s）后可重新处理

F15: Token 估算

场景：流式请求中途断开，上游未返回 usage。

估算规则：– 纯中文：`1.5 字符/token` – 纯英文：`4 字符/token` – 混合：按中文字符占比加权 – 输出 token：按已收到的 SSE chunk 内容长度估算

记录：`call_logs.usage_source = "estimated"` 标记为估算值。

Acceptance Criteria：– [ ] 流式断开 → `usage_source = "estimated"` – [ ] 正常结束 → `usage_source = "upstream"` – [ ] 估算值写入 `call_logs` 用于对账

F16: 预算告警

三级阈值：

阈值	触发条件	通知方式
50%	余额 ≤ 初始充值的 50%	管理后台首页黄色提示
80%	余额 ≤ 初始充值的 20%	管理后台红色提示 + Webhook
100%	余额 ≤ 0	管理后台红色警告 + Webhook + 拒绝新请求（402）

Webhook 通知：飞书 / 钉钉（管理员在设置中配置 URL） **检查频率**：后台定时任务，每 5 分钟检查一次所有 org 余额。

Acceptance Criteria：– [ ] 50% 阈值触发管理后台黄色提示 – [ ] 80% 阈值触发 Webhook 通知 – [ ] 100% 阈值拒绝新请求（402） – [ ] Webhook URL 可配置

F17: 系统设置

存储：`settings` 表（key–value，JSONB value）。

关键配置项：

Key	说明	默认值
`webhook.feishu`	飞书 Webhook URL	空
`webhook.dingtalk`	钉钉 Webhook URL	空
`sensitive_words`	敏感词库（JSON 数组）	[]
`backup.enabled`	自动备份开关	true
`backup.cron`	备份定时表达式	“0 2 * * *”

Acceptance Criteria： – [ ] 管理员可读写所有设置项 – [ ] 设置变更立即生效（无重启要求）

F18: 健康检查

端点	说明	响应
`GET /health/live`	存活检查	200 + `{"status":"ok"}`
`GET /health/ready`	就绪检查	200 + `{"status":"ok","db":"ok","redis":"ok"}` 或 503

Acceptance Criteria： – [ ] 服务启动后 `/health/live` 返回 200 – [ ] DB 或 Redis 不可用时 `/health/ready` 返回 503 – [ ] Docker HEALTHCHECK 使用 `/health/ready`

四、数据模型

表清单（14 张表）

表名	用途	关键约束
`orgs`	组织/租户	`name` 唯一

表名	用途	关键约束
`users`	用户	`(org_id, username)` 唯一
`api_keys`	API 密钥	`key_prefix` 唯一
`channels`	上游渠道	`status` 枚举，`priority` 排序
`channel_keys`	渠道 Key 池	`channel_id` 外键，`key_enc` 加密
`model_prices`	模型基础价格	`model_name` 唯一
`org_model_prices`	组织级定价覆盖	`(org_id, model_name)` 唯一
`wallets`	钱包	`org_id` 主键，`CHECK(balance>=0, frozen>=0)`
`transactions`	交易记录	`request_id` 索引
`call_logs`	调用日志	`request_id` 索引，`(org_id, created_at)` 复合索引
`audit_logs`	审计日志	Append-Only
`grants`	赠送记录	`org_id` 外键
`freeze_idempotency`	幂等性	`request_id` 主键
`settings`	系统设置	`key` 主键

完整字段定义见 `migrations/000001_init_schema.up.sql`。

五、非功能需求

维度	要求	验收方式
延迟	P95 < 500ms（不含上游延迟）	压测
并发	单实例 200 QPS	压测
数据安全	API Key AES-256-GCM 加密，余额 CHECK 约束	代码审查
可靠性	Docker `restart: always`，Redis AOF always	部署验证

维度	要求	验收方式
备份	每日 pg_dump 全量备份，保留 7 天	脚本验证
审计	所有管理操作写 `audit_logs`	代码审查
日志	结构化日志（zap），含 `request_id` / `org_id` / `model`	日志检查

六、部署架构

```
Docker Compose（3 个容器）：
├── uniroute（单个 Go 二进制）
│   ├── gateway/（路由转发，OneAPI 集成）
│   ├── billing/（计费模块）
│   ├── admin/（管理 API）
│   └── middleware/（鉴权、限流）
├── PostgreSQL 15
└── Redis 7（AOF always）
```

完整部署指南见 `ops/deployment-runbook.md`。

七、Phase 1 验收清单

核心链路

- [ ] 改 base_url 后可调用 DeepSeek / Qwen / GLM / GPT / Claude / Gemini
- [ ] 创建 API Key → 调用 API → 返回正确响应 + `_uniroute.cost`
- [ ] 充值 → 调用 → 余额正确扣减 → 对账一致
- [ ] 余额不足 → 返回 402 → 拒绝请求
- [ ] 禁用 Key → 调用 → 返回 401
- [ ] 流式请求完整返回 + 结算正确
- [ ] 流式请求中途断开 → 按已收到 token 结算

管理能力

- [ ] 管理后台可创建/编辑/禁用渠道
- [ ] 管理后台可创建/禁用 API Key
- [ ] 管理后台可查看账单和调用日志
- [ ] 管理后台可充值和调账
- [ ] Docker Compose 一键部署成功

稳定性

- [ ] 并发 100 请求，钱包余额不出现负数
- [ ] 渠道熔断后自动切换到备用渠道
- [ ] 敏感词请求返回 400
- [ ] 重复请求（相同 X-Request-Id）不重复扣费

八、明确不做的事情（Phase 1 排除）

功能	版本	说明
SaaS 多租户	v2	Phase 1 只做本地部署单租户
后付费	v2	Phase 1 只做预充值
代理商体系	v2	Phase 1 不做分销
语义缓存	v2	Phase 1 不做缓存
智能路由	v3	Phase 1 只做同模型重试
SSO / LDAP	v2	Phase 1 只做 JWT
2FA	v2	Phase 1 不做
Playground	v2	Phase 1 不做
内容审核 API	v2	Phase 1 只做 DFA 敏感词
工单系统	v2	Phase 1 不做
国际化	v2	Phase 1 仅中文
邮件通知	v2	Phase 1 只做 Webhook
跨模型 Fallback	不做	预冻结金额不匹配
Prompt / Response 存储	v2	Phase 1 不存储请求/响应内容

功能	版本	说明
License 校验	v2	Phase 1 合同约定
OpenTelemetry 集成	v3	Phase 1 不做

九、风险登记册

风险	概率	影响	缓解措施
计费预冻结公式不准	高	极高	修正公式 + 流式实时监控 + 安全系数 1.5
OneAPI Fork 深度耦合	高	高	接受 Deep Fork，完全放弃与上游同步
流式中断计费黑洞	高	极高	context.Canceled 强制结算 + 本地 Token 计数器
本地部署余额篡改	中	极高	v2 License 校验，v1 合同约定
模型厂商 API 变更	中	中	model_mapping + 版本化适配器
上游渠道宕机	中	高	熔断器 + 多渠道 fallback
客户 Key 泄露	中	高	IP 白名单 + 一键禁用
SSE 僵尸长连接	中	高	context.Canceled 监听 + 15s 心跳
重试风暴	中	高	全局重试 1 次，仅 5xx